

1. Project Overview

We implemented the distributed parameter server architecture from Li et al. (OSDI 2014) to train a logistic regression model on MNIST for binary even/odd classification. The model uses 785 weights (784 pixels + 1 bias). Parameter servers store weight shards partitioned via consistent hashing, while worker nodes compute gradients over disjoint data shards. Servers push updated weights back to workers after each round.

Features from the paper we implemented:

1. **Parameter server / worker** architecture on Ray. Servers own weight shards, workers own data shards, push/pull RPC interface. **Consistent hash ring** with virtual nodes (MD5-based) for load-balanced weight partitioning across servers
2. **Synchronization modes:**
 - a. BSP synchronization = all workers are on the same iteration step
 - b. Asynchronous synchronization = immediate gradient application, no coordination
 - c. Bounded delay (SSP) = ProgressTracker class enforces a staleness window between fastest and slowest worker
 - d. Experiments:
 - i. Accuracy vs. wall-clock time benchmarks — comparing convergence speed across sync modes
 - ii. Throughput across sync modes
3. **Fault tolerance strategies:**
 - a. Disk checkpointing = periodic JSON snapshots of each server's weight shard to disk
 - b. Chain replication = fire-and-forget weight replication to k clockwise ring neighbors after each update
 - c. Fault tolerance tests:
 - i. Static reshard — kill a server outside training, redistribute orphaned weights to survivors, verify correctness
 - ii. Live reshard — kill a server mid-training, recover weights, reconfigure ring and workers, resume training
 - d. Experiments: compared checkpoint vs chain replication recovery at different failure points

1. Architecture

2.1 Roles:

1. **Workers:** Each holds a data shard. On every step, they pull weights from servers, compute logistic regression gradients, and push gradient dicts back to the responsible servers. No direct worker-to-worker communication.
2. **Servers:** Own weight shards, buffer or immediately apply incoming gradients, and serve updated weights. Expose two RPCs — `pull_weights(indices, expected_iteration)` and

`push_gradients(gradient_dict, worker_id, iteration)`. Both roles are Ray remote actors, enabling concurrent execution.

Both workers and servers are Ray remote actors, enabling them to run concurrently across available CPU cores.

2.2 Consistent Hashing

With multiple servers, the system needs a deterministic, load-balanced rule for deciding which server is responsible for which weight indices. We implement a consistent hash ring (HashRing) using MD5, matching the approach described in the paper. Each server is assigned a configurable number of virtual node positions on a 2^{32} -bit hash ring and a weight index is mapped to the server whose nearest virtual node position is the first one clockwise.

The hash ring is built once at startup via `build_weight_map()`, which iterates over all weight indices and caches the server assignment in a dictionary. This weight map is passed to every worker at construction time so workers can route gradient pushes and weight pulls to the correct server without querying the ring on every step. When a server fails, `remove_server()` removes that server's virtual nodes and invalidates the cached map, triggering a rebuild that re-routes the orphaned weights to the remaining servers.

2.3 Data Sharding

Training data (MNIST) is randomly shuffled and split into `num_workers` equal shards, one per worker, so that each worker trains on a disjoint subset of examples throughout the entire training run. This is a standard data-parallel distribution strategy and mirrors the approach used by the paper.

2. Fault Tolerance

3.1 Strategy 1: Disk Checkpointing

In “checkpoint” mode, each `ParameterServer` periodically (based on `CHECKPOINT_EVERY` config parameter) serializes its weight shard and iteration number to a JSON file on “disk”.

The recovery procedure (`reshard_after_failure` in [recovery.py](#)) does the following:

- Reads the dead server's checkpoint
- Removes the dead server from the hash ring (`ring.remove_server()`)
- Determines new owners for each orphaned weight index by querying the updated ring
- Calls `absorb_weights()` on each surviving server to transfer the orphaned weights.
- Finally, all servers and workers are updated to agree on the latest committed iteration and the new ring topology.

3.2 Strategy 2: Chain Replication

In “chain” mode, we keep live copies of each server's weight shard on one or more neighboring servers in ring order. When a server fails, recovery simply reads the already-in-memory replica

from a surviving neighbor rather than going to disk. This means the recovered weights are current up to the last replication event, not just the last checkpoint.

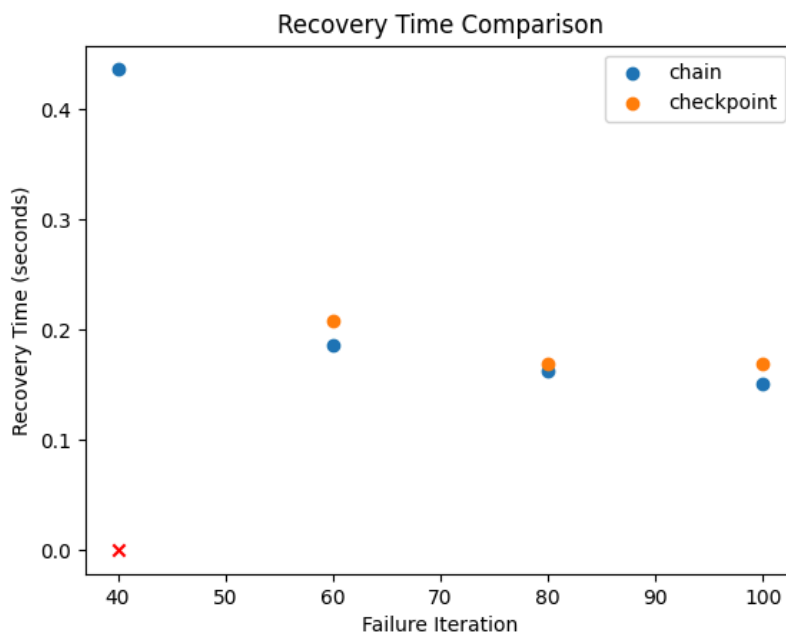
More specifically, each server stores received replica data in `replicated_shards`, a dict keyed by the leader server's ID. During recovery, `get_recovery_weights()` queries each surviving server for its replica of the dead server's shard until one is found.

3.3 Static and Live Reshard Tests

We implemented two tests for fault tolerance validation:

1. Static reshard test (`test_static_reshard`): kills a server outside of a training loop and verifies that (a) every orphaned weight lands on exactly one survivor with a value matching the last checkpoint, and (b) the union of all survivor shards covers all `NUM_WEIGHTS` weight indices exactly once.
2. Live reshard test (`run_live_reshard_trial`): kills a server mid-training at various iteration counts (40, 60, 80, 100), runs `reshard_after_failure`, resumes training for one more iteration, and measures accuracy before and after failure as well as total recovery time.

3.4 Recovery Time Comparison Between Checkpointing vs Chain Replication



The most notable result is that the checkpoint strategy fails if a server dies at iteration 40, because we only checkpoint every 50 iterations. This is the fundamental weakness of checkpointing: there is always a window of vulnerability between checkpoint writes.

For later failure points (60, 80, 100), both strategies recover in very similar time — roughly 0.15 to 0.21 seconds. The reason they are comparable in our setup is that our checkpoint files are very small (they contain only a JSON-serialized subset of weights, typically a few kilobytes), so

the “disk” read is fast. In a production deployment with billions of weight parameters, the checkpoint file would be gigabytes, and disk I/O would dominate recovery time.

3. Benchmarking different synchronization modes

One of the central contributions of the Li et al. paper is its flexibility around consistency models. Rather than mandating a single synchronization discipline, the framework lets the algorithm designer pick the right trade-off between convergence correctness and system throughput. We implement all three consistency modes described in the paper: BSP, Bounded Delay, and Asynchronous.

4.1 Bulk Synchronous Parallel

No worker starts iteration $t+1$ until every worker has completed iteration t and the servers have applied and committed all gradients for iteration t . This is the safest mode because every worker always trains on the same globally consistent weight vector, producing behavior identical to single-threaded gradient descent.

The cost of BSP is idleness because the fastest workers spend time waiting for the slowest.

4.2 Bounded Delay

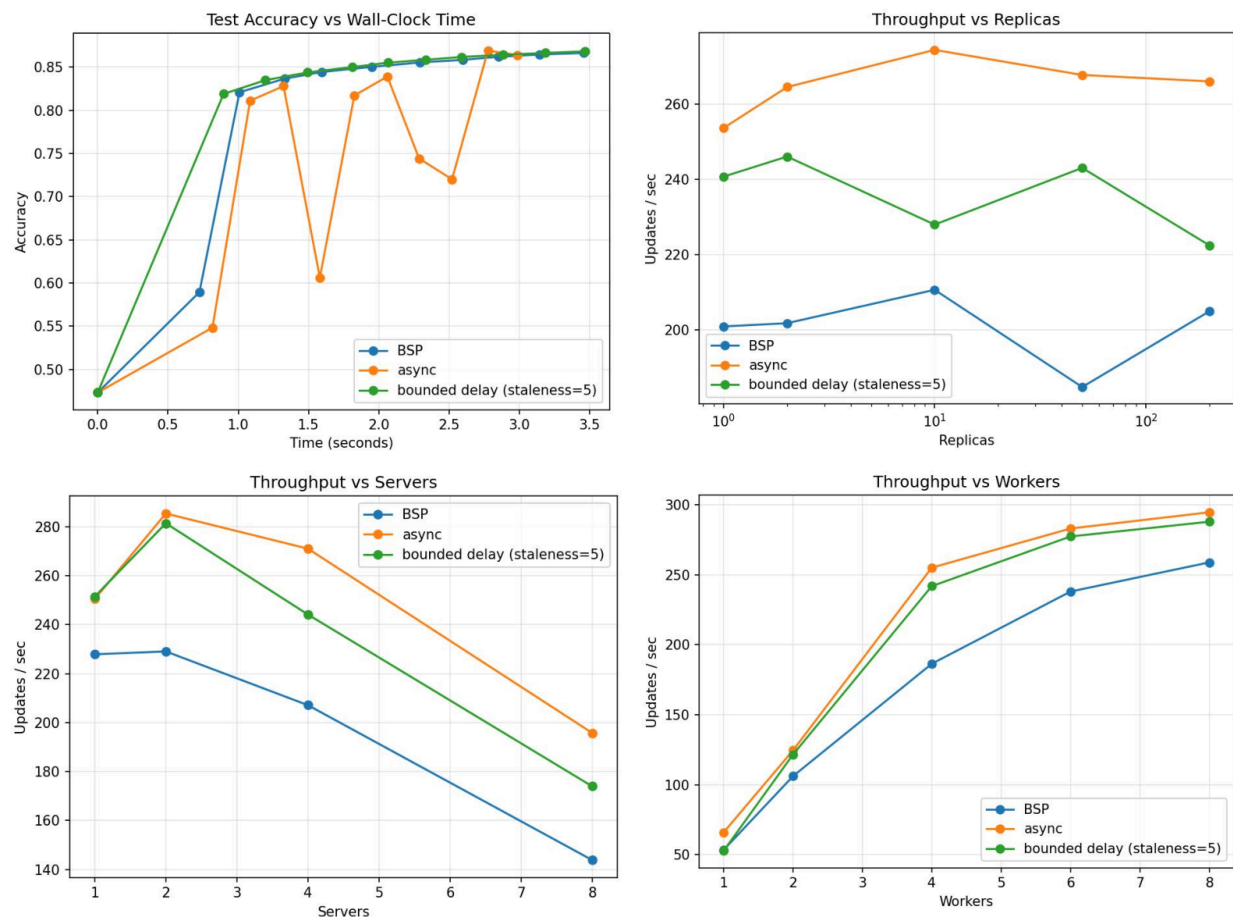
Bounded delay allows workers to run ahead by some set staleness window. We implement bounded delay through a shared ProgressTracker Ray actor. The tracker maintains a per-worker counter of completed steps and exposes two methods:

- `wait_until_can_advance(worker_id)`: blocks the calling worker until its step count is within the staleness bound of the minimum step count across all workers.
- `report_completed_step(worker_id)`: increments the worker's counter after completing a step.

4.3 Asynchronous

Workers operate fully independently and the server applies each gradient immediately upon receipt without waiting for other workers. The implementation is straightforward: in `push_gradients`, if `async_updates` is `True`, the server calls `_apply_gradients_immediate()` directly rather than buffering. In `train_loop_async`, workers run a tight loop of pull-compute-push without any iteration tracking or synchronization primitives.

4.4 Comparing the three modes



The plots above are a comparison of the three synchronization modes. The accuracy plot was generated using 6 workers, 2 servers, 2 ring replicas and test accuracy vs wall time over 100 training steps with eval every 10 steps. For the throughput plots, we used 100 gradient steps per worker for throughput. One subplot varies workers (servers fixed, ring replicas = 2), one varies servers (workers fixed), and one varies replicas on a log x-axis.

In the accuracy vs. time, we see that bounded delay and BSP converge stably to similar levels of accuracy in approximately the same amount of time. However, asynchronous training has much higher variance in accuracy. Since asynchronous training allows workers to pull and push updated weights at their own pace, stale reads and conflicting gradients can cause unstable convergence, resulting in the oscillating patterns in accuracy. However, overall, there is still a general trend towards greater accuracy.

For all three throughput graphs, asynchronous has the highest throughput, followed by bounded delay, and lastly BSP, as expected. On one end, asynchronous training has no blocks on making updates and maximizes parallelization, resulting in the most efficiency. On the other end, BSP requires all workers to be on the same iteration, meaning there is more overhead time due to blocking to ensure all workers have finished. Bounded delay offers a middle ground

between the efficiency of asynchronous and the stability of BSP. For all three synchronization modes, throughput is negatively correlated with the number of servers for more than 2 servers and positively correlated with the number of workers. This makes sense since more workers allows for greater parallelism in computing weights, but more servers means there is more overhead needed to pull from/push to each server. The number of replicas does not seem to have much of an effect on the throughput.